Distributed Tracing

spans, propagation, and which traces you keep

Watch the 4-min video

In one line: a trace is a tree of timed spans sharing one trace ID, stitched together by context propagated in headers; because keeping every trace is expensive you sample, and the choice between head and tail sampling decides whether you keep the slow and failed requests you actually need.

1 Two seconds, six hops, one slow call

A request takes 2 seconds. It touched a gateway, three services, a cache and a database. Somewhere one call is slow. Logs from six places with no common thread won't tell you which. A trace will, if you understand how it is built and how much of it you keep. (Lesson 36 covered tracing from the architecture side; this lesson goes inside the data.)

2 The data model: traces and spans

A **trace** is the whole journey of one request, identified by a 128-bit **trace ID**. It is made of **spans**: one timed operation each (an HTTP handler, an outgoing call, a DB query). A span records name, start, duration, status, attributes (e.g. `db.statement`, `http.response.status_code`), events, its own **span ID** and its **parent span ID**. Parent links turn spans into a tree, drawn as a waterfall.

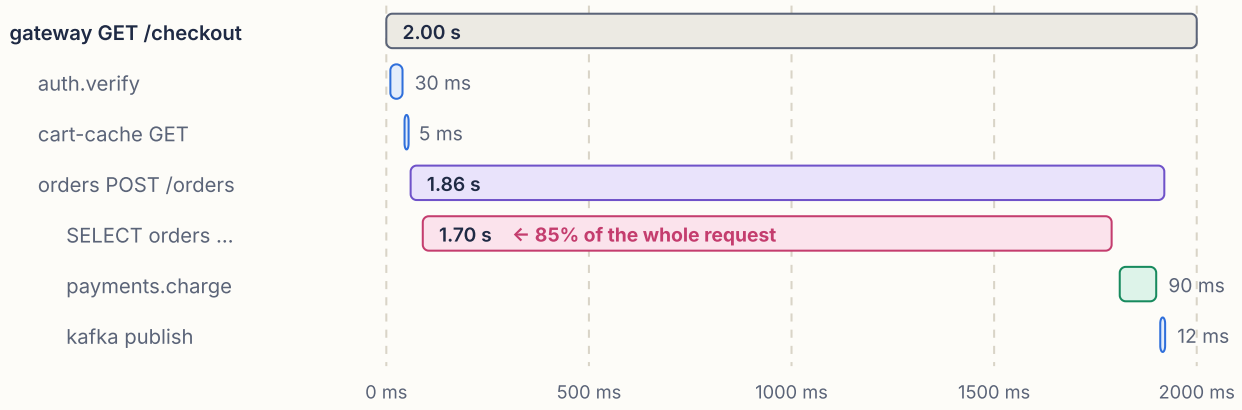


Figure 1. A trace waterfall. Indentation is the parent/child tree; bar length is duration. One DB span dominates the 2-second root span.

3 Context propagation

When a service calls another, it sends the trace ID and its current span ID in a header, standardised as the W3C **traceparent** header: `00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01` = version, trace ID (32 hex), parent span ID (16 hex), flags (`01` = sampled). The receiver starts a child span under that parent. Inside a process the current span lives in **context** (thread-local or async context), so instrumented libraries attach their spans automatically. Queues and background jobs need it carried in message headers (lesson 65).

4 Sampling: head vs tail

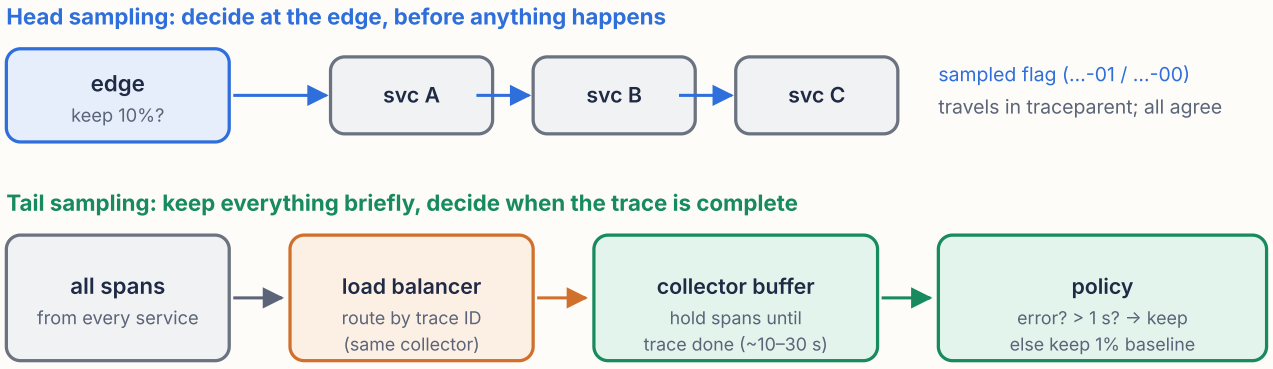


Figure 2. Head sampling decides blind but cheaply; tail sampling decides with full knowledge but must buffer every span, grouped by trace ID.

	Head sampling	Tail sampling
Decided	at the first span, before the outcome is known	after the trace completes
Keeps	a random X% (e.g. 10%)	errors, slow traces (> threshold), plus a small baseline
Cost	low: unsampled spans are never exported	high: every span sent to collectors and held in memory
Consistency	decision travels in the flags, all services agree	all spans of a trace must reach the same collector
Weakness	throws away most rare, interesting traces	more infrastructure; late spans can miss the decision

With 10% head sampling, an error that hits 0.1% of requests leaves you about one example trace per 10,000 requests. Tail sampling (e.g. the OpenTelemetry Collector's `tail_sampling` processor behind a `loadbalancing` exporter that routes by trace ID) keeps nearly all of them. Many teams combine both: light head sampling to cut volume, tail policies to keep what matters.

5 Trace backends

Open source: **Jaeger**, **Grafana Tempo** (object-storage based, cheap), Zipkin. Managed: Azure Monitor Application Insights, AWS X-Ray, Google Cloud Trace, Datadog, Honeycomb. Most accept OpenTelemetry data (lesson 64), so you can switch without re-instrumenting. Look for search by attribute, comparing slow and fast traces, service maps, and jumping from a span to its logs.

WORKED EXAMPLE: THE MISSING INDEX

A checkout request takes 2 s. In the waterfall (Figure 1) the root span is 2 s; most children are short, but one span, `SELECT … FROM orders WHERE customer_id = ?` from the orders service, takes **1.7 s**. Its attributes show the statement and table. `EXPLAIN` confirms a full table scan: `customer_id` has no index. After adding one, the span drops to **20 ms** and the request to about 300 ms.

COMMON MISTAKES

- One un-instrumented hop (a proxy, a queue) that drops the header: the trace splits into two unrelated traces.
- Head sampling at 1% and then wondering why there is no trace for the failed request.
- Putting full SQL with literal values or personal data into span attributes.
- Tail sampling without trace-ID-aware routing, so each collector sees only fragments.

6 Key terms

Trace: all spans of one request, sharing a trace ID

Span: one timed operation with parent link and attributes

traceparent: W3C header: version-traceid-parentid-flags

Head sampling: keep/drop decided at the start

Tail sampling: keep/drop decided after the trace completes

Waterfall: timeline view of a trace's span tree

7 Interview questions

Q1. What links spans into a trace? BASIC

A shared trace ID and parent span IDs, carried between services in propagation headers such as W3C traceparent.

Q2. Head vs tail sampling: what are the trade-offs? BASIC

Head sampling is cheap, simple and consistent but decides blindly, so it drops most slow and failed traces; tail sampling keeps errors and slow traces but needs collectors that buffer every span, routed by trace ID.

Q3. Traces show the frontend span but nothing downstream. Why? SCENARIO

Context isn't propagated: a hop (proxy, async queue, custom HTTP client) isn't instrumented or strips traceparent, so downstream spans start new traces.

Q4. Tracing every request costs too much. How do you cut cost without losing incidents? SCENARIO

Use tail sampling: keep all errors and requests above a latency threshold plus a 1–5% baseline; optionally add light head sampling to cut raw volume first.

Q5. Why must tail sampling route by trace ID? TRICKY

The decision needs the whole trace; if spans of one trace land on different collectors, none sees enough to decide, and the kept trace is incomplete.

Q6. A child span is longer than its parent. Is that a bug? TRICKY

Not necessarily: async work (fire-and-forget, a queued job) can outlive the parent; otherwise suspect clock skew between hosts.

Next: SD-N-006 · OpenTelemetry backendroadmap.com